Tarea 4 – Transformers

Clasificación de noticias Reuters con un codificador Transformer

Análisis completo del notebook: qué pedía la tarea, el enfoque, el código de cada celda, lo que se hizo y los resultados obtenidos

Diplomado en Machine Learning Avanzado

Módulo 5 – Unidad 4 · Deep Learning

Universidad Nacional de Colombia – Facultad de Ingeniería

Notebook: `v2 M5U4 Tarea 4.ipynb`

Documento elaborado el 7 de noviembre de 2026

Contenido

1. Qué pedía la tarea
2. El enfoque: un codificador Transformer con token [CLS]
3. Conceptos que se repiten en todo el notebook
4. Flujo general del notebook
5. Explicación celda por celda
    a. Configuración y calificador
    b. Carga de datos
    c. Punto 1 – Normalización de secuencias
    d. Punto 2 – Capa de codificación (token + posición)
    e. Punto 3 – Codificador
    f. Punto 4 – Construir el modelo
    g. Punto 5 – Entrenamiento
    h. Evaluación y envío
6. El hilo conductor: una noticia de principio a fin
7. Resultados y análisis
8. Lo que hubo que corregir para obtener el 100 %
9. Conclusiones

1. Qué pedía la tarea

La tarea plantea un problema de **clasificación de texto multiclase**: dado el contenido de una noticia de la agencia Reuters, asignarla a uno de **46 temas** (granos, petróleo, tasas de interés, adquisiciones, etc.). El requisito central del enunciado es que la clasificación se haga con un **Transformer** construido pieza por pieza: no se usa un modelo preentrenado, sino que el estudiante implementa el embedding con posición, el bloque codificador y el ensamblaje final como capas personalizadas de Keras.

Los datos

Se usa el dataset **Reuters newswire** incluido en Keras: 11.228 noticias etiquetadas, divididas en 8.982 de entrenamiento y 2.246 de validación. Cada noticia no llega como texto sino como una **lista de enteros** de longitud variable: cada número es el índice de una palabra, indexada por frecuencia (el 2 es la palabra más común del corpus, el 3 la segunda, etc.). El vocabulario se limita a las 20.000 palabras más frecuentes.

Los cinco puntos calificados

El notebook está organizado en cinco funciones que el estudiante debe completar. Cada una tiene su propia celda de prueba local y dos o tres celdas `grader.run_test(...)` que la envían al calificador automático de UNCode.

Punto	Función	Qué debe hacer
1	`seq_preprocess`	Llevar todas las secuencias a longitud 200, con relleno al final y el token [CLS] al inicio.
2	`get_EmbeddingLayer`	Capa personalizada que suma el embedding de cada palabra con el embedding de su posición.
3	`get_EncoderLayer`	Bloque codificador del Transformer: atención multi-cabeza + red feed-forward, con residuales,
4	`get_model`	Ensamblar el modelo, tomar la salida del [CLS] y compilar para clasificación multiclase.
5	`train_model`	Preprocesar los datos crudos y entrenar 10 épocas validando en cada una.

Cómo califica el grader

El calificador no se limita a revisar que el código "funcione". Ejecuta cada función con sus propias entradas y compara el resultado con una solución de referencia: en algunos tests compara **formas** (dimensiones de los arreglos), en otros **valores numéricos exactos** (el contenido de las secuencias preprocesadas, el número de parámetros) y en otros la **configuración interna** del modelo vía `model.get_config()`. Este detalle fue decisivo, como se verá en la sección 8: una función puede producir la forma correcta y aun así perder puntos por el contenido.

Al final, `grader.submit_task(globals())` vuelve a ejecutar todos los tests y registra la entrega oficial, visible en la página de la tarea en UNCode.

2. El enfoque: un codificador Transformer con token [CLS]

La arquitectura exigida es el **codificador** (encoder) del Transformer, propuesto en el artículo *Attention Is All You Need* (Vaswani et al., 2017). A diferencia de la Tarea 5 (que usa encoder y decoder para generar secuencias), aquí solo se necesita la mitad que "entiende": el objetivo no es generar texto sino producir **una sola etiqueta** por noticia.

> **Codificador (encoder).** Lee la noticia completa y produce, para cada palabra, un vector que ya incorpora información de las demás palabras. Es la parte que "entiende" la entrada.
>
> **Cabeza de clasificación.** Toma un único vector-resumen de la noticia y lo pasa por dos capas densas para decidir a cuál de los 46 temas pertenece. Es la parte que "decide".

El puente entre las dos partes es el token **[CLS]** (de *classification*, idea tomada de BERT): un token artificial que se antepone a toda secuencia. No corresponde a ninguna palabra real; es un "espacio en blanco" que, gracias a la atención, acumula información de toda la noticia al pasar por el codificador. Al final, en lugar de promediar las 200 salidas, basta tomar la salida de la posición 0 — la del [CLS] — como resumen vectorial de la noticia.

En esta tarea el modelo es deliberadamente pequeño: un solo bloque de encoder, embeddings de 16 dimensiones, 2 cabezas de atención y una red interna de 32 neuronas. Aun así suma 327.802 parámetros, casi todos (323.200) en la tabla de embeddings.

3. Conceptos que se repiten en todo el notebook

Token
La unidad mínima que procesa el modelo. Aquí cada palabra de la noticia es un token, representado por su índice entero.

Token especial [CLS]
Un token artificial (aquí con ID `1`) que se antepone a toda secuencia. Su salida tras el codificador se usa como resumen de la noticia completa para clasificar. El ID `0` queda reservado para el relleno.

Relleno (padding) y truncado
Las redes necesitan entradas de tamaño fijo. Las noticias cortas se completan con ceros al final (padding 'post') y a las largas se les recorta el inicio conservando el final (truncating 'pre'). Qué extremo se recorta fue la diferencia entre 50 y 100 en la sección 1.

Embedding
Tabla entrenable que convierte cada índice entero en un vector denso (aquí de 16 números). Al inicio los vectores son aleatorios; con el entrenamiento se ajustan para que palabras de significado parecido queden cerca.

Embedding de posición
La atención, por sí sola, no sabe en qué orden van los tokens. Se suma un segundo embedding que codifica la posición (0, 1, 2, ...) para que el modelo recupere la noción de orden. Aquí es aprendido, no el seno/coseno fijo del paper original.

Atención multi-cabeza (MultiHeadAttention)

Cada token calcula cuánto debe fijarse en cada uno de los demás y construye su nueva representación como una mezcla ponderada. Con varias "cabezas" (aquí 2) cada una puede especializarse en un tipo de relación distinto. Como la secuencia se mira a sí misma (`att(x, x)`), se llama **auto-atención**.

Conexión residual + normalización de capa

Tras cada sub-bloque se calcula `norm(x + salida)`. Sumar la entrada original evita que la información se pierda y facilita el flujo del gradiente; la normalización estabiliza la escala de los valores.

Dropout

Apaga neuronas al azar durante el entrenamiento para que el modelo no dependa de ninguna en particular y generalice mejor. En inferencia no actúa (por eso las capas propagan el argumento 'training').

Pérdida dispersa (sparse)

Las etiquetas son enteros (3, 4, 19, ...) y no vectores one-hot de 46 posiciones. Por eso corresponde `sparse_categorical_crossentropy`: la "forma dispersa" que menciona la nota del enunciado.

Conjuntos de entrenamiento y validación

El modelo ajusta sus parámetros con el conjunto de entrenamiento y se mide con el de validación, noticias que nunca vio. Las métricas `val_loss` y `val_accuracy` son las que informan si el modelo generaliza.

Semillas y reproducibilidad

Fijar las semillas aleatorias (Python, NumPy, Keras) en 0 hace que la inicialización de pesos, el dropout y el barajado den siempre lo mismo, de modo que las salidas puedan compararse con las esperadas del enunciado.

4. Flujo general del notebook

CÓDIGO

```
Reuters (11.228 noticias como listas de índices, longitud variable)
     |
     v
división ya hecha por Keras: 8.982 train / 2.246 validación
     |
     v
Punto 1  seq_preprocess    [CLS] + truncar/rellenar  ->  (n, 200)
     |
     v
Punto 2  get_EmbeddingLayer   token + posición  ->  (n, 200, 16)
     |
     v
Punto 3  get_EncoderLayer     atención + feed-forward  ->  (n, 200, 16)
     |
     v
Punto 4  get_model     tomar posición 0 ([CLS]) -> Dense(20, relu)
     |                 -> Dense(46, softmax)   ->  probabilidad por tema
     v
Punto 5  train_model   Adam 1e-4, sparse crossentropy, 10 épocas
     |
     v
~68-69 % de accuracy en validación  ->  submit_task
```

5. Explicación celda por celda

En esta sección se recorre el notebook en el orden exacto en que aparece. Para cada celda se indica qué hace, se muestra el código completo tal como quedó en el notebook, se explica línea por línea y se muestra la salida que se obtuvo al ejecutarla.

5a. Configuración y calificador

CELDA 1 Instalación del calificador

CÓDIGO

```
%%capture
!pip install rlxcrypt
!wget --no-cache -O session.pye -q
https://raw.githubusercontent.com/JuezUN/INGInious/master/external%20libs/session.pye
```

- `%%capture` es un comando mágico de Jupyter que oculta toda la salida de la celda.
- El signo `!` ejecuta un comando de terminal desde el notebook.
- `rlxcrypt` es la librería que descifra el módulo del calificador, que viene encriptado para que no se pueda leer cómo califica.
- `wget` descarga ese módulo con el nombre `session.pye`.

Salida: ninguna (está capturada). **Qué se hizo:** nada, la celda venía lista.

CELDA 2 Inicio de sesión en UNCode

CÓDIGO

```
import rlxcrypt
import session
grader = session.LoginSequence("DL-GroupMLDS-5-2026-2@3feead7a-0040-47fc-b701-11492b6ea6c8")
```

Crea el objeto `grader`, vinculado al curso y a la tarea mediante el identificador entre comillas. Al ejecutarse pide el usuario y la contraseña de UNCode. Todas las celdas que empiezan con `grader.` dependen de esta. **Qué se hizo:** nada; las credenciales las introduce la estudiante.

CELDA 3 Librerías y semillas

CÓDIGO

```
import keras
import numpy as np
import matplotlib.pyplot as plt
import os, random
%matplotlib inline
plt.style.use("ggplot")
# Seleccionamos las semillas para efectos de reproducibilidad
random.seed(0)
keras.utils.set_random_seed(0)
np.random.seed(0)
```

- `keras`: construcción y entrenamiento de redes neuronales. `numpy`: arreglos numéricos. `matplotlib`: gráficas (estilo ggplot).
- Las tres **semillas** fijan los generadores de números aleatorios de Python, Keras y NumPy. Los pesos de una red se inicializan al azar; con semilla fija salen siempre iguales. Como el enunciado compara salidas concretas (el summary, las curvas de entrenamiento), sin semillas nunca coincidirían.

CELDA 4 Versiones

CÓDIGO

```
# Versiones de las librerías usadas.
!python --version
print('NumPy', np.__version__)
print('keras', keras.__version__)
```

SALIDA

```
Python 3.13.16
NumPy 2.4.6
keras 3.13.2
```

Informativa. Las versiones del entorno son más recientes que las indicadas en el enunciado (Python 3.11.12, NumPy 2.0.2, Keras 3.8.0), pero todos los tests pasaron igual; solo las métricas de entrenamiento difieren en decimales (sección 7).

5b. Carga de datos

CELDA 5 Carga de Reuters y diccionario de palabras

CÓDIGO

```
vocab_size = 20000

(x_train, y_train), (x_val, y_val) = keras.datasets.reuters.load_data(
    num_words=vocab_size, index_from=2, path=0, start_char=None)
words = keras.datasets.reuters.get_word_index(path="reuters_word_index.json")
words = {valor: clave for clave, valor in words.items()}
print(len(x_train), "Training sequences")
print(len(x_val), "Validation sequences")
```

SALIDA

```
8982 Training sequences
2246 Validation sequences
```

- `num_words=20000`: conserva solo las 20.000 palabras más frecuentes; cualquier otra se descarta de las secuencias.
- `index_from=2`: desplaza los índices reales de palabra para reservar los IDs bajos. Gracias a esto el **0 queda libre para el relleno** y el **1 para el token [CLS]** — es la razón de que en el punto 1 se use `cls_id = 1` sin chocar con ninguna palabra.
- `start_char=None`: evita que Keras anteponga su propio token de inicio a cada secuencia, porque el [CLS] lo vamos a agregar nosotros en el punto 1.
- `get_word_index` devuelve el diccionario palabra → índice; la última línea lo **invierte** (índice → palabra) para poder reconstruir el texto de una noticia.

CELDA 6 Ver un ejemplo

CÓDIGO

```
print(f"texto: {' '.join([words[i] for i in x_train[0]])}")
```

SALIDA

```
texto: ... mcgrath rentcorp said as a result of its december acquisition of space co it
expects earnings per share in 1987 of 1 15 to 1 30 dlrs per share up from 70 cts in 1986
... said pretax net should rise to nine to 10 mln dlrs from six mln dlrs ... reuter 3
(texto reconstruido de la primera noticia; se muestra un fragmento)
```

Recorre la primera noticia índice por índice, busca cada palabra en el diccionario invertido y las une con espacios. Sirve para comprobar que los datos cargaron bien y para ver cómo lucen las noticias: texto financiero en minúsculas, sin puntuación, con números separados. **Qué se hizo:** nada; es una celda de inspección.

5c. Punto 1 – Normalización de secuencias

Qué se pedía. Normalizar las secuencias para que todas tengan la misma longitud (`max_len`), añadiendo el token `[CLS]` y contando ese token dentro del tamaño final. El enunciado da dos consideraciones explícitas: el [CLS] cuenta dentro del tamaño de la secuencia, y **el relleno debe ir en la parte FINAL**. Salida: arreglo NumPy de tamaño (n, max_len).

CELDA 7 Función calificada seq_preprocess

CÓDIGO

```
# FUNCIÓN CALIFICADA seq_preprocess
def seq_preprocess(X, cls_id, max_len):
    # Truncamos/rellenamos cada secuencia a max_len-1 (reservando espacio para [CLS]).
    # Relleno al FINAL (padding='post'); el truncado conserva la parte final
    # de la secuencia (truncating='pre', comportamiento por defecto).
    X_pad = keras.utils.pad_sequences(X, maxlen=max_len - 1,
                                      padding='post', truncating='pre', value=0)
    # Anteponemos el token [CLS] a todas las secuencias
    cls_col = np.full((X_pad.shape[0], 1), cls_id)
    X_out = np.hstack([cls_col, X_pad])
    return X_out
```

Línea por línea:

- `pad_sequences(X, maxlen=max_len-1, ...)`: lleva cada noticia a 199 posiciones. Se usa `max_len - 1` y no `max_len` porque hay que **reservar un espacio para el [CLS]**, de modo que el total sea exactamente 200.
- `padding='post'`: si la noticia es **corta**, se completan con ceros las posiciones del **final** — exactamente lo que exige el enunciado ('el relleno debe usarse para completar la secuencia en la parte FINAL').
- `truncating='pre'`: si la noticia es **larga**, se recorta por el **inicio** y se conserva la parte final. Es el comportamiento por defecto de Keras y el que espera el calificador (ver sección 8: aquí estuvo el error que costó 50 puntos).
- `value=0`: el relleno usa el ID 0, que quedó reservado gracias a `index_from=2` y no corresponde a ninguna palabra.
- `np.full((n, 1), cls_id)`: crea una columna de n filas llena del ID del [CLS] (unos).
- `np.hstack([cls_col, X_pad])`: pega esa columna **al frente** del arreglo. Toda fila queda como `[1, w1, w2, ..., 0, 0]`. Importante: el [CLS] se agrega **después** de truncar; si se antepusiera antes, el truncado 'pre' se lo comería en las noticias largas.

CELDA 8 Celda de prueba

CÓDIGO

```
# TEST CELL
max_len = 200

x_train_p = seq_preprocess(x_train, 1, max_len = max_len)
x_val_p = seq_preprocess(x_val, 1, max_len = max_len)

print(f'Tamaño de la particion de entrenamiento:{x_train_p.shape}')
print(f'Tamaño de la particion de validacion:{x_val_p.shape}')
```

SALIDA

```
Tamaño de la particion de entrenamiento:(8982, 200)
Tamaño de la particion de validacion:(2246, 200)
```

Coincide con la salida esperada del enunciado. Nótese que aquí se fija `cls_id = 1`: el [CLS] es el entero 1 en todas las secuencias.

CELDA 9 Evaluación del punto 1 (tres tests)

CÓDIGO

```
grader.run_test("Test 1_1", globals())
grader.run_test("Test 1_2", globals())
grader.run_test("Test 1_3", globals())
```

SALIDA

```
Test 1_1   Test Run correctly   Your test grade: 100
Test 1_2   Test Run correctly   Your test grade: 100
Test 1_3   Test Run correctly   Your test grade: 100
```

El Test 1_1 verifica formas y casos sencillos; los Tests 1_2 y 1_3 comparan el **contenido exacto** de los arreglos, incluyendo noticias más largas que 199 palabras. En la primera versión de la función estos dos tests dieron 50; la sección 8 cuenta el diagnóstico completo.

5d. Punto 2 – Capa de codificación (token + posición)

Qué se pedía. Crear una capa personalizada de Keras que entregue, para cada token, su representación (embedding) **más** la codificación posicional. Entradas: `max_len`, `vocab_size`, `embed_dim`. Salida: una capa de Keras lista para usar.

CELDA 10 Función calificada get_EmbeddingLayer

CÓDIGO

```
        # FUNCIÓN CALIFICADA get_EmbeddingLayer
        def get_EmbeddingLayer(max_len, vocab_size, embed_dim):

            class TokenAndPositionEmbedding(keras.layers.Layer):
                def __init__(self, max_len, vocab_size, embed_dim, **kwargs):
                    super().__init__(**kwargs)
                    self.max_len = max_len
                    self.vocab_size = vocab_size
                    self.embed_dim = embed_dim
                    self.token_emb = keras.layers.Embedding(input_dim=vocab_size,
                                                            output_dim=embed_dim)
                    self.pos_emb = keras.layers.Embedding(input_dim=max_len,
                                                          output_dim=embed_dim)

                def call(self, x):
                    maxlen = keras.ops.shape(x)[-1]
                    positions = keras.ops.arange(start=0, stop=maxlen, step=1)
                    positions = self.pos_emb(positions)
                    x = self.token_emb(x)
                    return x + positions

                def get_config(self):
                    config = super().get_config()
                    config.update({"max_len": self.max_len,
                                   "vocab_size": self.vocab_size,
                                   "embed_dim": self.embed_dim})
                    return config

            layer = TokenAndPositionEmbedding(max_len, vocab_size, embed_dim)
            return layer
```

Línea por línea:

- Se hereda de `keras.layers.Layer` para crear una **capa personalizada**. En `__init__` se llama a `super().__init__(**kwargs)` (obligatorio para que Keras registre la capa) y se guardan los tres hiperparámetros como atributos.
- `token_emb`: tabla de 20.000 × 16 — una fila (vector entrenable) por cada palabra del vocabulario. Responde a "*qué palabra es*".
- `pos_emb`: tabla de 200 × 16 — una fila por cada posición posible. Responde a "*dónde está*". Es un embedding posicional **aprendido**, a diferencia del seno/coseno fijo del paper original.
- En `call`: `keras.ops.shape(x)[-1]` lee la longitud de la secuencia (200) y `keras.ops.arange(0, maxlen)` genera el vector `[0, 1, ..., 199]`. Se usan las operaciones de `keras.ops` (y no NumPy) para que funcione dentro del grafo de cómputo con cualquier backend.
- `return x + positions`: la **suma** de ambos embeddings es la representación final. El broadcasting de la suma aplica el mismo vector de posiciones a todas las noticias del batch.
- `get_config`: devuelve los hiperparámetros para que la capa sea serializable. Es lo que hace que `model.get_config()` (celda 16) muestre `max_len`, `vocab_size` y `embed_dim`, tal como aparece en la salida esperada del punto 4.

CELDA 11 Celda de prueba

CÓDIGO

```
        # TEST_CELL
        max_len = 200
        emb_layer = get_EmbeddingLayer(max_len, vocab_size, 128)
        codified_x = emb_layer(x_train_p[:10])
        print(f'Codificación de sequencias de entrenamiento: {codified_x.shape}')
```

SALIDA

```
        Codificación de sequencias de entrenamiento: (10, 200, 128)
```

Se prueba con 10 secuencias y embeddings de 128: cada entero de la matriz (10, 200) se convirtió en un vector de 128 números → tensor (10, 200, 128). Coincide con lo esperado.

CELDA 12

Evaluación del punto 2 (tres tests)

CÓDIGO

```
        grader.run_test("Test 2_1", globals())
        grader.run_test("Test 2_2", globals())
        grader.run_test("Test 2_3", globals())
```

SALIDA

```
        Test 2_1   Test Run correctly   Your test grade: 100
        Test 2_2   Test Run correctly   Your test grade: 100
        Test 2_3   Test Run correctly   Your test grade: 100
```

5e. Punto 3 – Codificador

Qué se pedía. Construir una capa personalizada que implemente el bloque codificador de un Transformer. Entradas: `embed_dim`, `num_heads` (cabezas de atención), `ff_dim` (tamaño de la red interna) y `rate` (tasa de Dropout). Salida: la capa.

CELDA 13 **Función calificada get_EncoderLayer**

CÓDIGO

```
        # FUNCIÓN CALIFICADA get_EncoderLayer
        def get_EncoderLayer(embed_dim, num_heads, ff_dim, rate):
            class TransformerBlock(keras.layers.Layer):
                def __init__(self, embed_dim, num_heads, ff_dim, rate, **kwargs):
                    super().__init__(**kwargs)
                    self.embed_dim = embed_dim
                    self.num_heads = num_heads
                    self.ff_dim = ff_dim
                    self.rate = rate
                    self.att = keras.layers.MultiHeadAttention(num_heads=num_heads,
                                                               key_dim=embed_dim)
                    self.ffn = keras.Sequential([
                        keras.layers.Dense(ff_dim, activation="relu"),
                        keras.layers.Dense(embed_dim),
                    ])
                    self.layernorm1 = keras.layers.LayerNormalization(epsilon=1e-6)
                    self.layernorm2 = keras.layers.LayerNormalization(epsilon=1e-6)
                    self.dropout1 = keras.layers.Dropout(rate)
                    self.dropout2 = keras.layers.Dropout(rate)

                def call(self, inputs, training=None):
                    attn_output = self.att(inputs, inputs)
                    attn_output = self.dropout1(attn_output, training=training)
                    out1 = self.layernorm1(inputs + attn_output)
                    ffn_output = self.ffn(out1)
                    ffn_output = self.dropout2(ffn_output, training=training)
                    return self.layernorm2(out1 + ffn_output)

                def get_config(self):
                    config = super().get_config()
                    config.update({"embed_dim": self.embed_dim,
                                   "num_heads": self.num_heads,
                                   "ff_dim": self.ff_dim,
                                   "rate": self.rate})
                    return config

            layer = TransformerBlock(embed_dim, num_heads, ff_dim, rate)
            return layer
```

Los componentes, en el orden en que se ejecutan dentro de `call`:

- **1. Auto-atención:** `self.att(inputs, inputs)` — al pasar la misma entrada como *query* y como *value*, cada token atiende a todos los tokens de su propia secuencia. `num_heads=2` cabezas aprenden patrones de atención distintos en paralelo; `key_dim=embed_dim` fija el tamaño de las proyecciones internas.
- **2. Dropout 1:** apaga el 10 % (`rate=0.1`) de las activaciones **solo durante el entrenamiento** — por eso `call` recibe y propaga el argumento `training`.
- **3. Residual + LayerNorm 1:** `layernorm1(inputs + attn_output)` — la suma deja pasar la señal original (facilita el gradiente) y la normalización estabiliza las escalas. `epsilon=1e-6` evita divisiones por cero.

- **4. Feed-forward:** `Dense(ff_dim, relu) -> Dense(embed_dim)` aplicada a cada posición por separado: expande a 32 dimensiones, aplica la no linealidad y regresa a 16 para poder sumar el residual.
- **5. Dropout 2 + Residual + LayerNorm 2:** mismo patrón, cerrando el bloque. La salida conserva la forma de la entrada: (batch, 200, embed_dim).

CELDA 14 Celda de prueba

CÓDIGO

```
# TEST_CELL
encoder_layer = get_EncoderLayer(128, 2, 32, .1)
encoded_x = encoder_layer(codified_x)
print(f'Codificación de sequencias de entrenamiento: {encoded_x.shape}')
```

SALIDA

```
Codificación de sequencias de entrenamiento: (10, 200, 128)
```

La forma no cambia respecto a la entrada — es la gracia del bloque: transforma las representaciones (ahora cada token "sabe" de los demás) sin alterar las dimensiones, lo que permitiría apilar varios bloques en serie.

CELDA 15 Evaluación del punto 3 (dos tests)

CÓDIGO

```
grader.run_test("Test 3_1", globals())
grader.run_test("Test 3_2", globals())
```

SALIDA

```
Test 3_1   Test Run correctly   Your test grade: 100
Test 3_2   Test Run correctly   Your test grade: 100
```

5f. Punto 4 – Construir el modelo

Qué se pedía. Completar `get_model` ensamblando las estructuras anteriores. Instrucciones explícitas del enunciado: usar el token **[CLS]** para obtener la representación final después del codificador, y emplear la activación, función de pérdida y métrica adecuadas para **clasificación multiclase**. La nota final pide decidir si debe usarse la **forma dispersa** de la pérdida según la estructura de las etiquetas.

CELDA 16 **Función calificada get_model**

CÓDIGO

```
        # FUNCIÓN CALIFICADA get_model
        def get_model(max_len, vocab_size, embed_dim, num_heads, ff_dim, rate,
                      num_clases, learning_rate, n_neurons):
            # Fijamos una semilla para efectos de reproducibiidad
            np.random.seed(0)
            keras.utils.set_random_seed(0)
            keras.backend.clear_session()

            inputs = keras.layers.Input(shape=(max_len,))
            x = get_EmbeddingLayer(max_len, vocab_size, embed_dim)(inputs)
            x = get_EncoderLayer(embed_dim, num_heads, ff_dim, rate)(x)
            # Representación del token [CLS] (primera posición)
            x = x[:, 0, :]
            x = keras.layers.Dense(n_neurons, activation="relu")(x)
            outputs = keras.layers.Dense(num_clases, activation="softmax")(x)

            model = keras.Model(inputs=inputs, outputs=outputs)
            model.compile(optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
                          loss="sparse_categorical_crossentropy",
                          metrics=["accuracy"])
            return model
```

Línea por línea:

- Las tres primeras líneas (semillas + `clear_session()`) venían dadas: reinician el estado de Keras y fijan el azar para que el modelo salga idéntico en cada ejecución, con los mismos nombres de capa (`dense_2`, `dense_3`) que la salida esperada.
- `Input(shape=(max_len,))`: el modelo recibe secuencias ya preprocesadas de 200 enteros.
- Se encadenan las capas de los puntos 2 y 3 con la API funcional de Keras: cada capa se crea y se aplica inmediatamente sobre el tensor anterior.
- `x = x[:, 0, :]` — **la línea más importante**: de las 200 salidas del codificador toma solo la posición 0, la del token [CLS], como resumen de la noticia. En el summary aparece como la capa `get_item (GetItem)` con salida (None, 16) y 0 parámetros, exactamente como en la salida esperada.
- Cabeza de clasificación: `Dense(n_neurons, relu)` como capa intermedia (20 neuronas) y `Dense(num_clases, softmax)` como salida — softmax convierte los 46 puntajes en una distribución de probabilidad que suma 1 (la activación correcta para multiclase con una sola etiqueta).
- **La decisión de la nota del enunciado:** las etiquetas `y_train` son enteros (3, 4, 19, ...), no vectores one-hot → corresponde `sparse_categorical_crossentropy`. Si fueran one-hot, sería `categorical_crossentropy`. Métrica: `accuracy`.
- Optimizador `Adam` con la tasa de aprendizaje recibida como parámetro (1e-4 en las pruebas).

CELDA 17 Celda de prueba: summary del modelo

CÓDIGO

```
# TEST_CELL
max_len = 200
vocab_size = 20000
embed_dim = 16
num_heads = 2
ff_dim = 32
rate = 0.1
num_clases = 46
learning_rate = 1e-4
n_neurons = 20

model = get_model(max_len, vocab_size, embed_dim, num_heads, ff_dim, rate,
                  num_clases, learning_rate, n_neurons)
model.summary()
```

SALIDA

```
Model: "functional_1"
| Layer (type)                        | Output Shape      |   Param # |
| input_layer (InputLayer)            | (None, 200)       |         0 |
| token_and_position_embedding        | (None, 200, 16)   |   323,200 |
| transformer_block                   | (None, 200, 16)   |     3,296 |
| get_item (GetItem)                  | (None, 16)        |         0 |
| dense_2 (Dense)                     | (None, 20)        |       340 |
| dense_3 (Dense)                     | (None, 46)        |       966 |
 Total params: 327,802 (1.25 MB)
 Trainable params: 327,802 (1.25 MB)
 Non-trainable params: 0 (0.00 B)
```

La cuenta de parámetros coincidió **exactamente** con la salida esperada y se puede verificar a mano:

- Embedding: 20.000 × 16 (tokens) + 200 × 16 (posiciones) = 320.000 + 3.200 = **323.200**.
- Bloque transformer: proyecciones de la atención + red feed-forward (16→32→16) + dos LayerNorm = **3.296**.
- Densas: 16×20 + 20 (sesgos) = **340**; 20×46 + 46 = **966**. Total: **327.802**.

CELDA 18 Celda de prueba: configuración interna

CÓDIGO

```
import pprint

pprint.pprint(model.get_config())
```

SALIDA

```
{'input_layers': [['input_layer', 0, 0]],
 'layers': [{'class_name': 'InputLayer', ...},
            {'class_name': 'TokenAndPositionEmbedding',
             'config': {..., 'embed_dim': ..., 'max_len': 200,
                        'vocab_size': 20000}, ...},
            {'class_name': 'TransformerBlock',
             'config': {..., 'embed_dim': ..., 'ff_dim': 32,
                        'num_heads': 2, 'rate': 0.1}, ...},
            {'class_name': 'Dense', 'config': {'activation': 'relu', ...}},
            {'class_name': 'Dense', 'config': {'activation': 'softmax', ...}}],
 'name': 'functional_1', 'output_layers': [['dense_3', 0, 0]], ...}
```

Esta celda existe para verificar la **estructura interna**: que las capas personalizadas expongan sus hiperparámetros vía `get_config` (por eso lo implementamos en los puntos 2 y 3) y que las

activaciones sean relu y softmax. Es la forma en que el grader puede revisar el diseño del modelo sin entrenarlo.

CELDA 19

Evaluación del punto 4 (dos tests)

CÓDIGO

```
grader.run_test("Test 4_1", globals())
grader.run_test("Test 4_2", globals())
```

SALIDA

```
Test 4_1   Test Run correctly   Your test grade: 100
Test 4_2   Test Run correctly   Your test grade: 100
```

5g. Punto 5 – Entrenamiento

> **Qué se pedía.** Completar `train_model`, que recibe el modelo compilado y las secuencias **crudas** (de longitud variable) de entrenamiento y validación, junto con `cls_id`, `max_len` y `epochs`. Debe preprocesar y ajustar el modelo, devolviendo el objeto `history` con las métricas por época.

CELDA 20 — Función calificada train_model

CÓDIGO

```
# FUNCIÓN CALIFICADA train_model
def train_model(model, cls_id, X_train, Y_train, X_val, Y_val, max_len, epochs):
    X_train_p = seq_preprocess(X_train, cls_id, max_len)
    X_val_p = seq_preprocess(X_val, cls_id, max_len)
    history = model.fit(X_train_p, Y_train,
                        validation_data=(X_val_p, Y_val),
                        epochs=epochs, batch_size=32)
    return history
```

Línea por línea:

- Las dos primeras líneas aplican `seq_preprocess` (punto 1) a entrenamiento y validación con el **mismo** `cls_id` y `max_len`: el modelo debe ver en validación datos con idéntico formato al de entrenamiento.
- `model.fit(...)`: entrena. `batch_size=32` es el valor que reproduce la salida esperada: 8.982 ÷ 32 ≈ **281 pasos por época**, el `281/281` que aparece en los logs. Un batch distinto habría delatado el error de inmediato.
- `validation_data=(X_val_p, Y_val)`: al final de cada época se evalúan `val_accuracy` y `val_loss` sobre datos que el modelo no usa para ajustar pesos.
- Se retorna `history`, que guarda las cuatro métricas época por época (útil para graficar curvas de aprendizaje).

CELDA 21 — Celda de prueba: entrenar 10 épocas

CÓDIGO

```
max_len = 200
epochs = 10
cls_id = 1
model = get_model(max_len, vocab_size, embed_dim, num_heads, ff_dim, rate,
                  num_clases, learning_rate, n_neurons)
history = train_model(model, cls_id, x_train, y_train, x_val, y_val, max_len, epochs)
```

SALIDA

```
Epoch 1/10
281/281 - accuracy: 0.3233 - loss: 2.9602 - val_accuracy: ~0.49 - val_loss: ~2.4
...
Epoch 9/10
281/281 - accuracy: ~0.72  - val_accuracy: 0.6897 - val_loss: 1.3090
Epoch 10/10
281/281 - accuracy: 0.7383 - loss: 1.0753 - val_accuracy: 0.6861 - val_loss: 1.3119
```

El modelo pasa de ~32 % de accuracy en la primera época a **73,8 % en entrenamiento** y **68,6 % en validación** en la décima. La referencia del enunciado terminaba en val_accuracy 0.6808 y val_loss 1.3179 — nuestra corrida quedó incluso ligeramente mejor; la diferencia de decimales se debe a las versiones más nuevas de Keras/Python del entorno (sección 7).

CELDA 22 **Evaluación del punto 5 (dos tests)**

CÓDIGO

```
grader.run_test("Test 5_1", globals())
grader.run_test("Test 5_2", globals())
```

SALIDA

```
Test 5_1   (entrena internamente 10 épocas)   Test Run correctly   Your test grade: 100
Test 5_2   (entrena internamente 10 épocas)   Test Run correctly   Your test grade: 100
```

Estos dos tests son los más lentos (~30 y ~60 segundos con GPU): cada uno construye un modelo y lo entrena desde cero llamando a nuestras funciones. Tras corregir `seq_preprocess` se volvieron a ejecutar para confirmar que seguían en 100 con el preprocesamiento nuevo.

5h. Evaluación y envío

CELDA 23 **Entrega oficial**

CÓDIGO

```
grader.submit_task(globals())
```

SALIDA

```
Grading succeeded
TEST Test 1_1 , Your test grade: 100     TEST Test 1_2 , Your test grade: 100
TEST Test 1_3 , Your test grade: 100     TEST Test 2_1 , Your test grade: 100
TEST Test 2_2 , Your test grade: 100     TEST Test 2_3 , Your test grade: 100
TEST Test 3_1 , Your test grade: 100     TEST Test 3_2 , Your test grade: 100
TEST Test 4_1 , Your test grade: 100     TEST Test 4_2 , Your test grade: 100
TEST Test 5_1 , Your test grade: 100     TEST Test 5_2 , Your test grade: 100
```

La entrega vuelve a ejecutar los 12 tests en el servidor (incluidos los entrenamientos) y registra la submission en UNCode. **Resultado final: 12/12 tests en 100.** Hubo una submission anterior con los tests 1_2 y 1_3 en 50, hecha antes de la corrección del truncado.

6. El hilo conductor: una noticia de principio a fin

Para amarrar todas las piezas, este es el recorrido de una noticia concreta (supongamos una de 500 palabras sobre resultados financieros) a través del sistema completo:

- **Entrada.** La noticia llega como una lista de 500 enteros: `[3767, 282, ..., 17, 12]`.
- **Punto 1 (`seq_preprocess`).** Como 500 > 199, se descartan las primeras 301 palabras y se conservan las últimas 199 (el cierre de la noticia, donde suelen estar las cifras y conclusiones). Se antepone el 1 del [CLS]: queda una fila de 200 enteros que empieza por 1 y no tiene relleno. Una noticia de 80 palabras, en cambio, quedaría `[1, w1, ..., w80, 0, 0, ..., 0]` con 119 ceros al final.
- **Punto 2 (embedding).** Cada uno de los 200 enteros se convierte en un vector de 16 números (su embedding de palabra) al que se suma el embedding de su posición. La noticia es ahora una matriz 200 × 16 donde cada fila sabe qué palabra es y dónde está.
- **Punto 3 (codificador).** La auto-atención deja que cada palabra se redefina mirando a las otras 199 — "interest" cerca de "rates" no significa lo mismo que cerca de "public interest" — y la fila del [CLS], que no significaba nada, absorbe información de toda la noticia. La matriz sigue siendo 200 × 16, pero ahora cada fila es contextual.
- **Punto 4 (modelo).** `x[:, 0, :]` toma solo la fila del [CLS] (16 números que resumen la noticia), la pasa por 20 neuronas relu y por la softmax de 46 salidas: el tema con mayor probabilidad es la predicción.
- **Punto 5 (entrenamiento).** El proceso se repite 10 épocas sobre las 8.982 noticias en lotes de 32, ajustando los 327.802 parámetros para minimizar la entropía cruzada; tras cada época se mide sobre las 2.246 noticias de validación que el modelo nunca ve al entrenar.

7. Resultados y análisis

Calificación

Sección	Tests	Primera versión	Versión final
1. Normalización (seq_preprocess)	1_1 / 1_2 / 1_3	100 / 50 / 50	100 / 100 / 100
2. Embedding token + posición	2_1 / 2_2 / 2_3	100 / 100 / 100	100 / 100 / 100
3. Bloque codificador	3_1 / 3_2	100 / 100	100 / 100
4. Construcción del modelo	4_1 / 4_2	100 / 100	100 / 100
5. Entrenamiento	5_1 / 5_2	100 / 100	100 / 100

La entrega final terminó con **"Grading succeeded" y los 12 tests en 100**.

Comportamiento del entrenamiento

- La accuracy de entrenamiento y la de validación suben **juntas** durante las 10 épocas (de ~32 % a 73,8 % y de ~49 % a 68,6 % respectivamente): el modelo seguía aprendiendo, sin señales fuertes de sobreajuste — la brecha final entre ambas es de ~5 puntos.

- Para dimensionar el 68,6 % de validación: con 46 clases, el azar daría ~2 % y predecir siempre la clase mayoritaria ~36 %. El modelo casi duplica esa línea base, lo que confirma que la atención extrae señal temática real.
- La val_loss seguía bajando en la época 10 (1.3119), lo que sugiere que con más épocas, un embedding mayor o más bloques de encoder habría margen de mejora. No se hizo porque los tests fijan los hiperparámetros.
- Las cifras no son idénticas a las del enunciado (val_accuracy 0.6808 / val_loss 1.3179) porque el Colab actual trae Python 3.13 y Keras 3.13.2 en lugar de Python 3.11 y Keras 3.8.0; la semilla fija el azar del código propio pero no las diferencias internas entre versiones ni el no-determinismo de la GPU. El grader lo tolera: evalúa con sus propios márgenes.

8. Lo que hubo que corregir para obtener el 100 %

La primera versión de `seq_preprocess` anteponía el [CLS] y luego truncaba/rellenaba con `truncating='post'` — es decir, conservaba el **inicio** de las noticias largas y recortaba su final:

CÓDIGO

```
        # PRIMERA VERSIÓN (tests 1_2 y 1_3: 50/100)
        def seq_preprocess(X, cls_id, max_len):
            X_cls = [[cls_id] + list(x) for x in X]
            X_out = keras.utils.pad_sequences(X_cls, maxlen=max_len,
                                              padding='post', truncating='post', value=0)
            return np.array(X_out)
```

El diagnóstico, paso a paso:

- El Test 1_1 dio 100 — verifica la forma del arreglo y los casos de noticias cortas, donde ambas versiones coinciden (en una noticia corta no se trunca nada).
- Los Tests 1_2 y 1_3 dieron **50** y su feedback mostraba el arreglo esperado contra el obtenido. En las filas correspondientes a noticias **largas**, el arreglo esperado terminaba en índices de palabras reales distintos de los nuestros, y sus primeras palabras tras el [CLS] no eran las primeras de la noticia sino unas de la mitad.
- Esa huella identifica el comportamiento: la referencia conserva la **parte final** de la secuencia (truncado 'pre', que además es el valor por defecto de Keras), no la inicial. El enunciado solo fijaba el lado del **relleno** ('en la parte FINAL'); el lado del truncado había que deducirlo del feedback.
- La corrección tiene un matiz importante: no basta cambiar 'post' por 'pre' en la versión original, porque con el [CLS] ya antepuesto un truncado 'pre' **eliminaría el propio [CLS]** en las noticias largas. La solución es truncar/rellenar primero las secuencias a max_len−1 y anteponer el [CLS] **después**, con np.hstack.
- Tras el cambio se reejecutaron la función, la celda de prueba y los tests: 1_1, 1_2 y 1_3 pasaron a 100. También se reejecutaron 5_1 y 5_2 (que usan seq_preprocess internamente al entrenar) para confirmar que seguían en 100, y se envió la submission final.

> **Moraleja.** En preprocesamiento de texto, *qué extremo de la secuencia se descarta* es una decisión de diseño con consecuencias: aquí valía 50 puntos. Y en noticias de agencia tiene sentido conservar el final — los cables de Reuters suelen cerrar con las cifras y la síntesis del tema. Cuando un test falla parcialmente, el feedback (esperado vs. obtenido) contiene casi siempre la pista exacta; leerlo con cuidado evita adivinar.

9. Conclusiones

- Se construyó un clasificador Transformer **completo y desde cero** en Keras: preprocesamiento con [CLS], embedding doble (token + posición), bloque codificador con atención multi-cabeza, residuales, LayerNorm y Dropout, y cabeza densa con softmax.
- El pipeline respeta las decisiones que exigía el enunciado: relleno al final, [CLS] contado dentro de max_len, pérdida en forma dispersa por etiquetas enteras, y batch de 32 que reproduce los 281 pasos por época.
- El modelo alcanzó **~68,6 % de accuracy de validación** en 46 clases con solo 327.802 parámetros y 10 épocas, superando ampliamente las líneas base de azar (~2 %) y clase mayoritaria (~36 %), y ligeramente la referencia del enunciado (68,1 %).
- La calificación final fue **12/12 tests en 100** y la submission quedó registrada en UNCode con "Grading succeeded".
- La lección central del proceso fue metodológica: el detalle del **truncado** (conservar el final y no el inicio de las noticias largas, y anteponer el [CLS] después de truncar) era la única diferencia entre 50 y 100 en la sección 1, y se resolvió leyendo el feedback del calificador en lugar de probar a ciegas.